Construindo uma API de Reservas com Node.js e um Aplicativo Mobile com Expo

Um Guia Completo para o Desenvolvimento Full-Stack

Título: Construindo uma API de Reservas com Node.js e um Aplicativo Mobile com Expo
Subtítulo: Um Guia Completo para o Desenvolvimento Full-Stack
Autor: Roginaldo Franco

Sumário:

- Introdução
- Capítulo 1 — Preparação do Ambiente
  - Requisitos
  - Instalação do Backend
  - Instalação do App
  - Executando o Backend
  - Executando o App
  - Configuração de IP
  - Problemas Comuns
- Capítulo 2 — Arquitetura do Sistema
  - Visão Geral
  - Backend
  - Frontend
  - Fluxo de Comunicação
  - Estrutura do Backend
- Capítulo 3 — Servidor (app.ts)
  - Arquivo Completo
  - Explicação Detalhada
- Capítulo 4 — Banco de Dados (SQLite)
  - Arquivo Completo
  - Explicação Detalhada: SQLite, Seed e Bcrypt
  - (Imagem: Diagrama simples de tabelas `usuarios` e `reservas` com suas relações)
- Capítulo 5 — Autenticação (JWT)
  - Middleware Completo
  - Explicação Detalhada: O Poder do JWT
  - (Imagem: Ilustração do fluxo de autenticação JWT com token circulando)
- Capítulo 6 — Rotas de Autenticação
  - Arquivo Completo
  - Explicação Detalhada: Cadastro e Login
- Capítulo 7 — Rotas de Usuário
  - Arquivo Completo
  - Explicação Detalhada: Perfil do Usuário
- Capítulo 8 — Rotas de Reservas
  - Arquivo Completo
  - Explicação Detalhada: Criando e Listando Reservas
- Capítulo 9 — Aplicativo Mobile (Expo)
  - Configuração da API
  - Tela de Login
  - Tela de Cadastro
  - Tela Principal (Home)
  - Consumo Protegido
  - (Imagem: Capturas de tela simuladas das telas de Login, Cadastro e Home do aplicativo mobile)
- Capítulo 10 — Fluxo Completo de Ponta a Ponta
  - O Ciclo de Vida de uma Requisição

    - (Imagem: Fluxograma detalhado do fluxo de dados desde o app até o backend e vice-versa)
- Capítulo 11 — Testes Básicos
    - Testando o Login com `curl`
- Capítulo 12 — Evolução e Próximos Passos
    - Sugestões de Melhoria
- Conclusão
- Apêndice — Tipos TypeScript

Introdução

O desenvolvimento de sistemas modernos exige, cada vez mais, a integração entre diferentes camadas de software. Não basta compreender apenas o backend ou apenas o frontend: é necessário entender como essas partes se comunicam, como os dados fluem e como decisões de arquitetura impactam o sistema como um todo.

Este livro foi concebido com esse objetivo. Ao longo dos capítulos, o leitor construirá um sistema completo de reservas, composto por uma API desenvolvida em Node.js e um aplicativo mobile construído com Expo (React Native). Trata-se de um projeto intencionalmente simples, mas suficientemente realista para introduzir conceitos fundamentais utilizados em sistemas profissionais.

Diferentemente de materiais que apresentam apenas trechos isolados de código, aqui o foco está na compreensão do processo. Cada decisão será contextualizada, cada arquivo terá um propósito claro e cada etapa contribuirá para a construção de um sistema funcional.

Ao final, o leitor terá não apenas um projeto executável, mas também uma base sólida para evoluir para arquiteturas mais complexas. O objetivo não é apenas apresentar código, mas explicar de forma progressiva como cada parte do sistema funciona e como elas se conectam.

Ao final deste livro, você será capaz de:

- Criar uma API RESTful robusta com Node.js.
- Implementar autenticação segura utilizando JSON Web Tokens (JWT).
- Trabalhar com bancos de dados relacionais locais com SQLite.
- Desenvolver aplicativos mobile interativos com Expo (React Native).
- Compreender o fluxo completo de comunicação e autenticação entre backend e frontend.

Capítulo 1 — Preparação do Ambiente

Antes de iniciarmos a codificação, é fundamental preparar o terreno. Uma configuração de ambiente correta evita dores de cabeça futuras e garante que o desenvolvimento flua sem interrupções.

(Imagem: Ícones representando Node.js, npm, VS Code e Terminal)

Requisitos:

Para acompanhar este material de forma eficaz, certifique-se de que seu ambiente possua:

- Node.js (versão 16 ou superior): A base para a execução do nosso backend JavaScript.
- npm (Node Package Manager): Essencial para gerenciar as dependências do projeto.
- Um editor de código: Recomendamos o Visual Studio Code (VS Code) por sua vasta gama de extensões úteis para desenvolvimento.
- Acesso a um terminal: Seja PowerShell, Bash, Zsh ou outro de sua preferência, ele será seu principal aliado para executar comandos.

O Expo, por sua vez, será utilizado via `npx`, o que significa que não é necessário instalá-lo globalmente.

Instalação do Backend:

Navegue até a raiz do diretório do projeto onde os arquivos do backend se encontram. Em seguida, abra seu terminal e execute o seguinte comando:

Bash

```bash
npm install
```

Este comando lerá o arquivo `package.json` e instalará todas as dependências necessárias, como o framework Express para o servidor, o driver do SQLite para o banco de dados e bibliotecas para autenticação.

Instalação do App Mobile:

O código do aplicativo mobile está organizado dentro da pasta `app/app-reservas`. Acesse esta pasta no seu terminal e execute o mesmo comando de instalação de dependências:

Bash

```bash
cd app/app-reservas
npm install
```

Executando o Backend:

Com as dependências instaladas, é hora de levantar nosso servidor. Na raiz do projeto, execute:

Bash

```bash
npm run dev
```

Se tudo ocorrer como esperado, você verá uma mensagem indicando que o servidor está ativo e disponível em:

```
Servidor rodando em http://localhost:3000
```

Executando o App Mobile:

Agora, na pasta do aplicativo (`app/app-reservas`), inicie o Expo com o seguinte comando. O parâmetro `-c` é importante, pois força a limpeza do cache, o que pode resolver problemas de inconsistência durante o desenvolvimento:

Bash

```bash
npx expo start -c
```

O Expo iniciará um servidor de desenvolvimento e apresentará um QR Code no terminal. Você pode escanear este código com o aplicativo Expo Go no seu dispositivo físico ou usar um emulador.

Configuração de Rede (Importante!):

Ao testar o aplicativo em um dispositivo físico, não é possível acessar `localhost` diretamente. Para que o aplicativo mobile se comunique com o backend, você precisará utilizar o endereço IP local da máquina onde o servidor está rodando.

Abra o arquivo:

`app/lib/services/api.ts`

E substitua a linha:

TypeScript

```
export const API_URL = "http://localhost:3000";
```

Por:

TypeScript

```
export const API_URL = "http://<seu_ip_local>:3000";
```

Substitua `<seu_ip_local>` pelo endereço IP da sua máquina na rede local (por exemplo, `192.168.1.10`).

Problemas Comuns:

- O servidor não inicia: Verifique se as dependências foram instaladas corretamente e se a versão do Node.js é compatível.
- O app não conecta ao backend: Confirme se o IP configurado em `api.ts` está correto e se seu dispositivo mobile está na mesma rede Wi-Fi que o servidor.
- Erro de requisição: Certifique-se de que o servidor backend esteja realmente rodando e respondendo.

Resolver esses pequenos contratempos faz parte do aprendizado e solidifica sua compreensão do ambiente de desenvolvimento.

Capítulo 2 — Arquitetura do Sistema

Antes de mergulharmos no código, é crucial ter uma visão clara da estrutura do nosso sistema. Compreender a arquitetura nos ajuda a navegar pelo projeto com mais confiança e a tomar decisões de desenvolvimento mais assertivas.

(Imagem: Um diagrama de blocos mostrando a separação entre Backend e Frontend, com setas indicando a comunicação via API)

Visão Geral:

Nosso sistema é composto por duas partes principais, que trabalham em conjunto para oferecer uma experiência completa ao usuário:

1. Backend (API): O cérebro do sistema.
2. Frontend (Aplicativo Mobile): A interface com o usuário.

Backend:

O backend é o guardião dos dados e das regras de negócio. Suas responsabilidades incluem:

- Gerenciar dados: Armazenar, recuperar e atualizar informações no banco de dados.
- Validar regras de negócio: Garantir que as operações realizadas estejam em conformidade com as lógicas definidas (ex: uma sala só pode ser reservada em um determinado horário).
- Controlar autenticação e autorização: Verificar a identidade dos usuários e seus permissões de acesso.
- Responder às requisições do cliente: Processar as solicitações vindas do aplicativo mobile e enviar as respostas adequadas.

Ele funciona como a "fonte da verdade" do sistema, garantindo a integridade e a consistência das informações.

Frontend:

O aplicativo mobile é a "cara" do nosso sistema. Sua função é proporcionar a interação com o usuário:

- Interação com o usuário: Apresentar interfaces intuitivas e coletar informações.
- Coleta de dados: Capturar as entradas do usuário (ex: detalhes de uma reserva).
- Exibição de informações: Apresentar os dados recebidos do backend de forma clara e organizada.
- Comunicação com a API: Enviar requisições para o backend e processar as respostas.

Fluxo de Comunicação:

O funcionamento do sistema segue um padrão simples e eficiente:

1. O usuário interage com o aplicativo mobile: Realiza uma ação (ex: tenta fazer uma reserva).
2. O aplicativo faz uma requisição HTTP para a API: Envia os dados necessários para o backend.
3. A API processa a requisição: Valida os dados, interage com o banco de dados e aplica as regras de negócio.
4. A resposta retorna ao aplicativo: O backend envia o resultado da operação (sucesso, erro, dados solicitados).
5. O aplicativo atualiza a interface: Exibe uma mensagem de sucesso, um erro ou os dados atualizados para o usuário.

Este ciclo é repetido continuamente durante o uso do sistema, garantindo uma experiência fluida e responsiva.

Estrutura do Backend:

Para manter nosso código organizado, escalável e de fácil manutenção, adotamos uma estrutura de diretórios baseada em responsabilidades:

```
src/
├── app.ts            # Ponto de entrada da aplicação, configuração do servidor
├── database/         # Configuração e lógica de acesso ao banco de dados
├── middlewares/      # Funções intermediárias (ex: autenticação)
├── routes/           # Definição das rotas da API (endpoints)
└── types/            # Definições adicionais do TypeScript (se aplicável)
```

Cada diretório desempenha um papel específico:

- `app.ts`: O coração da aplicação backend, onde o servidor Express é configurado, os middlewares são aplicados e as rotas são registradas.
- `database/`: Contém a lógica para inicializar o banco de dados, definir esquemas e realizar operações de leitura/escrita.
- `middlewares/`: Arquivos que encapsulam funcionalidades que interceptam requisições antes de chegarem aos controladores de rota (ex: verificação de token de autenticação).
- `routes/`: Define os diferentes endpoints da API, agrupando rotas por funcionalidade (ex: `auth.routes.ts`, `reserva.routes.ts`).
- `types/`: Pode ser usado para definir interfaces e tipos customizados em TypeScript, melhorando a segurança e legibilidade do código.

Essa estrutura modular facilita a localização de funcionalidades, a adição de novas features e a colaboração em equipe, permitindo que o sistema evolua sem se tornar um emaranhado de código.

Capítulo 3 — Servidor (app.ts)

O arquivo `app.ts` é o ponto de partida da nossa aplicação backend. É nele que configuramos o servidor Express, definimos middlewares essenciais e registramos todas as rotas que compõem nossa API.

Arquivo Completo:

TypeScript

```typescript
import express from "express";
import cors from "cors";

// Importa a configuração do banco de dados para garantir sua inicialização
import "./database/database";

// Importa os módulos de rotas
import authRoutes from "./routes/auth.routes";
import usuarioRoutes from "./routes/usuario.routes";
import reservaRoutes from "./routes/reserva.routes";

const app = express();

// --- Middlewares ---
// cors(): Permite que requisições de origens diferentes (como o app mobile) sejam aceitas.
app.use(cors());
// express.json(): Transforma o corpo das requisições recebidas em formato JSON em objetos JavaScript
acessíveis.
app.use(express.json());
```

```
// --- Registro de Rotas ---
// Agrupa as rotas por domínio para melhor organização.
app.use("/auth", authRoutes); // Rotas relacionadas à autenticação (login, cadastro)
app.use("/usuarios", usuarioRoutes); // Rotas relacionadas a informações de usuários
app.use("/reservas", reservaRoutes); // Rotas para gerenciar reservas

// --- Rota de Verificação (Ping) ---
// Uma rota simples para verificar se a API está online.
app.get("/", (_, res) => {
  res.send("API rodando");
});

app.get("/ping", (_, res) => {
  res.json({ pong: true });
});

// --- Inicialização do Servidor ---
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});

export default app; // Exporta a instância do app para possíveis testes ou integrações
```

Explicação Detalhada:

1. Importações:
   - `express`: O framework web que utilizaremos para construir nossa API.
   - `cors`: Um middleware que habilita o Cross-Origin Resource Sharing, permitindo que nosso aplicativo mobile (rodando em uma origem diferente) faça requisições para a API.
   - `./database/database`: Importar este arquivo garante que a configuração e inicialização do nosso banco de dados SQLite ocorram antes que qualquer rota seja definida ou executada.
   - `./routes/...`: Importamos os módulos de rotas que criamos separadamente para organizar o código. Cada arquivo de rota (`auth.routes.ts`, `usuario.routes.ts`, `reserva.routes.ts`) conterá a lógica específica para um conjunto de endpoints.
2. Instância do Express:
   - `const app = express();`: Criamos uma instância do aplicativo Express, que será o ponto central do nosso servidor.
3. Middlewares:
   - `app.use(cors());`: Este middleware é aplicado globalmente. Ele configura o servidor para aceitar requisições vindas de qualquer origem. Em cenários de produção, é comum configurar o `cors` para permitir apenas origens específicas por segurança.
   - `app.use(express.json());`: Outro middleware global. Ele é responsável por parsear o corpo das requisições que chegam com o `Content-Type` definido como `application/json`. Isso significa que se enviarmos um JSON em uma requisição POST ou PUT, ele estará disponível em `req.body` como um objeto JavaScript. Sem ele, `req.body` seria `undefined`.
4. Registro de Rotas:

- `app.use("/auth", authRoutes);`: Aqui, estamos dizendo ao Express que todas as rotas definidas no arquivo `auth.routes.ts` devem ser prefixadas com `/auth`. Por exemplo, se `authRoutes` tiver uma rota `/login`, o endpoint completo será `/auth/login`. Isso ajuda a organizar os endpoints da API de forma lógica.
- `app.use("/usuarios", usuarioRoutes);` e `app.use("/reservas", reservaRoutes);`: Mesma lógica aplicada para as rotas de usuários e reservas.

5. Rota de Verificação (`/` e `/ping`):
   - `app.get("/", (_, res) => { res.send("API rodando"); });`: Uma rota de nível raiz simples que retorna uma mensagem de texto. Útil para verificar rapidamente se a API está online.
   - `app.get("/ping", (_, res) => { res.json({ pong: true }); });`: Uma rota mais convencional para testes de "ping". Ela retorna um objeto JSON `{ pong: true }`, que é uma prática comum para verificar a saúde de APIs. O _ é usado para indicar que o parâmetro `req` (requisição) não será utilizado nesta função.
6. Inicialização do Servidor:
   - `app.listen(PORT, () => { ... });`: Este método inicia o servidor HTTP. Ele "escuta" por requisições na porta especificada (neste caso, 3000). A função de callback é executada quando o servidor inicia com sucesso, e exibimos uma mensagem no console informando o endereço onde a API está disponível.

Este arquivo é a espinha dorsal da nossa API. Com ele configurado, temos um servidor rodando, pronto para receber e responder às requisições do nosso aplicativo mobile.

Capítulo 4 — Banco de Dados (SQLite)

Para armazenar as informações de usuários e reservas, utilizaremos o SQLite. Ele é uma excelente escolha para projetos de pequeno e médio porte, pois é um banco de dados embarcado, sem a necessidade de um servidor separado, e é muito simples de configurar.

Arquivo Completo:

TypeScript

```
import Database from "better-sqlite3";
import bcrypt from "bcryptjs"; // Para criptografar senhas

// Inicializa o banco de dados SQLite. Se o arquivo 'banco.db' não existir, ele será criado.
const db = new Database("banco.db");

// Cria as tabelas 'usuarios' e 'reservas' se elas ainda não existirem.
// A sintaxe SQL é executada diretamente no banco de dados.
db.exec(`
CREATE TABLE IF NOT EXISTS usuarios (
  id INTEGER PRIMARY KEY AUTOINCREMENT, -- Chave primária autoincrementável
  nome TEXT NOT NULL,                   -- Nome do usuário, não pode ser nulo
  email TEXT UNIQUE NOT NULL,           -- Email do usuário, deve ser único e não nulo
  senha TEXT NOT NULL                   -- Senha do usuário (será armazenada como hash), não pode ser nula
);

CREATE TABLE IF NOT EXISTS reservas (
  id INTEGER PRIMARY KEY AUTOINCREMENT, -- Chave primária autoincrementável
  sala TEXT NOT NULL,                   -- Nome ou identificador da sala reservada
  data TEXT NOT NULL,                   -- Data da reserva (armazenada como texto, pode ser formatada como
YYYY-MM-DD)
  usuario_id INTEGER,                   -- Chave estrangeira referenciando o usuário que fez a reserva
```

```
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) -- Define a relação com a tabela 'usuarios'
);
`);

// --- Seed de Dados (Dados Iniciais) ---
// Verificamos se um usuário administrador já existe para evitar duplicação.
const adminEmail = "admin@example.com";
const getUser = db.prepare("SELECT id FROM usuarios WHERE email = ?"); // Prepara uma consulta para buscar
o ID do usuário pelo email

let adminId = (getUser.get(adminEmail) as any)?.id; // Executa a consulta e obtém o ID, se o usuário
existir

// Se o administrador ainda não existe, criamos um.
if (!adminId) {
  // Criptografa a senha com bcrypt. O '8' é o custo computacional (salt rounds).
  const senhaHash = bcrypt.hashSync("admin123", 8);
  // Insere o novo usuário administrador no banco de dados.
  const result = db
    .prepare("INSERT INTO usuarios (nome, email, senha) VALUES (?, ?, ?)")
    .run("Admin", adminEmail, senhaHash); // Executa a inserção com os dados do administrador

  adminId = result.lastInsertRowid; // Obtém o ID do registro recém-criado
}

// Exporta a instância do banco de dados para ser usada em outros módulos.
export default db;
```

Explicação Detalhada: SQLite, Seed e Bcrypt

(Imagem: Um diagrama simples mostrando as tabelas `usuarios` e `reservas`. A tabela `usuarios` com colunas `id`, `nome`, `email`, `senha`. A tabela `reservas` com colunas `id`, `sala`, `data`, `usuario_id`. Uma seta conectando `reservas.usuario_id` a `usuarios.id` indicando a chave estrangeira.)

- SQLite e `better-sqlite3`:
  - Nossa escolha para o banco de dados é o SQLite, conhecido por sua simplicidade e por não requerer um servidor de banco de dados separado. Ele armazena todo o banco de dados em um único arquivo (`banco.db` neste caso).
  - Utilizamos o pacote `better-sqlite3` para interagir com o SQLite de forma eficiente e com uma sintaxe mais próxima ao JavaScript.
- Criação de Tabelas (`db.exec`):
  - O método `db.exec()` permite executar múltiplos comandos SQL de uma vez.
  - `CREATE TABLE IF NOT EXISTS usuarios (...)`: Cria a tabela `usuarios` apenas se ela ainda não existir.
    - `id INTEGER PRIMARY KEY AUTOINCREMENT`: Define `id` como a chave primária, um número inteiro que se incrementa automaticamente a cada novo registro.
    - `nome TEXT NOT NULL`: Um campo de texto para o nome do usuário, que não pode ser deixado em branco.
    - `email TEXT UNIQUE NOT NULL`: O email do usuário, que deve ser único em toda a tabela e também é obrigatório.
    - `senha TEXT NOT NULL`: O campo para a senha. Armazenaremos aqui o *hash* da senha, nunca a senha em texto plano.
  - `CREATE TABLE IF NOT EXISTS reservas (...)`: Cria a tabela `reservas`.

        - `sala TEXT NOT NULL, data TEXT NOT NULL`: Campos para a sala e a data da reserva.
        - `usuario_id INTEGER`: Um campo que armazenará o ID do usuário que fez a reserva.
        - `FOREIGN KEY (usuario_id) REFERENCES usuarios(id)`: Esta é a definição da chave estrangeira. Ela garante que o `usuario_id` em `reservas` deve corresponder a um `id` existente na tabela `usuarios`. Isso ajuda a manter a integridade referencial dos dados.
- Seed de Dados (Dados Iniciais):
    - O "seed" é o processo de popular o banco de dados com dados iniciais. Isso é especialmente útil para testes rápidos e para garantir que o sistema tenha um ponto de partida.
    - Neste caso, verificamos se um usuário com o email `admin@example.com` já existe. Se não existir, criamos um registro para um usuário administrador.
    - `db.prepare("SELECT id FROM usuarios WHERE email = ?")`: O método `prepare` é usado para criar consultas parametrizadas. Isso é mais seguro e eficiente, pois o banco de dados pode otimizar a execução. O `?` é um placeholder que será substituído pelo valor passado para `.get()`.
    - `getUser.get(adminEmail)`: Executa a consulta preparada e retorna a primeira linha que corresponde aos critérios (ou `undefined` se nenhum registro for encontrado).
- Criptografia de Senhas com Bcrypt:
    - `(bcrypt)": O pacote` bcryptjs` é uma implementação do algoritmo bcrypt, amplamente recomendado para hashear senhas. Ele adiciona um "salt" aleatório a cada senha antes de hashear, tornando os ataques de força bruta muito mais difíceis.
    - `bcrypt.hashSync("admin123", 8)`: Gera um hash da senha "admin123". O segundo argumento (8) é o "custo" ou número de rodadas de hashing. Um valor maior aumenta a segurança, mas também o tempo de processamento.
    - Importante: Nunca armazene senhas em texto plano! Sempre use um algoritmo de hashing forte como o bcrypt.

Ao final deste capítulo, temos um banco de dados configurado e pronto para armazenar as informações essenciais do nosso sistema.

Capítulo 5 — Autenticação (JWT)

A autenticação é um pilar fundamental de qualquer aplicação que gerencia dados de usuários. Nosso sistema utilizará JSON Web Tokens (JWT) para gerenciar a autenticação de forma segura e eficiente.

Middleware Completo:

TypeScript

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
```

```
// Definição de uma chave secreta. Em produção, esta chave deve ser complexa e mantida em segredo.
// Idealmente, deve ser carregada de variáveis de ambiente.
const SEGREDO = "segredo_super_simples_deve_ser_trocado";

/**
 * Middleware de autenticação para verificar a validade do token JWT.
 * Espera o token no cabeçalho 'Authorization' no formato 'Bearer TOKEN'.
 */
export function auth(req: Request, res: Response, next: NextFunction) {
  // 1. Obter o cabeçalho de autorização da requisição.
  const authHeader = req.headers.authorization;

  // 2. Verificar se o cabeçalho existe. Se não, o usuário não está autenticado.
  if (!authHeader) {
    return res.status(401).json({ erro: "Token não enviado" });
  }

  // 3. Dividir o cabeçalho para extrair o token. O formato esperado é "Bearer TOKEN".
  // O split(' ') retorna um array: ['Bearer', 'TOKEN']. Pegamos o segundo elemento.
  const [, token] = authHeader.split(" ");

  // 4. Tentar verificar a validade do token usando a chave secreta.
  try {
    // jwt.verify retorna o payload do token decodificado se for válido.
    const decoded = jwt.verify(token, SEGREDO) as any;
    // Adiciona o ID do usuário decodificado à requisição para que as rotas subsequentes possam acessá-lo.
    req.usuarioId = decoded.id;
    // Chama o próximo middleware ou a função do controlador da rota.
    next();
  } catch (error) {
    // Se a verificação falhar (token inválido, expirado, etc.), retorna um erro 401.
    return res.status(401).json({ erro: "Token inválido" });
  }
}
```

Explicação Detalhada: O Poder do JWT

(Imagem: Uma ilustração do fluxo de autenticação JWT. Um usuário envia credenciais para o servidor, o servidor verifica e retorna um token. Em requisições subsequentes, o cliente envia o token no cabeçalho Authorization, e o servidor o valida para conceder acesso.)

- O que é JWT?
  JSON Web Token (JWT) é um padrão aberto (RFC 7519) que define uma maneira compacta e autônoma de transmitir informações entre partes como um objeto JSON. Essas informações podem ser verificadas e confiáveis porque são assinadas digitalmente. Um JWT consiste em três partes separadas por pontos (`.`): um cabeçalho, um payload e uma assinatura.
- Vantagens do JWT para Autenticação:
  - Stateless (Sem Estado): O servidor não precisa armazenar informações de sessão do usuário. Toda a informação necessária para verificar a autenticidade está contida no próprio token. Isso facilita a escalabilidade, pois qualquer instância do servidor pode validar o token.
  - Autocontido: O token contém informações sobre o usuário (como seu ID), permitindo que o servidor tome decisões de autorização sem consultar o banco de dados a cada requisição.
  - Simplicidade: A autenticação via token é relativamente simples de implementar em ambos os lados (backend e frontend).

- Como o Middleware Funciona:
    - Obtenção do Token: O middleware `auth` intercepta a requisição e procura pelo cabeçalho `Authorization`. O padrão comum é que o cliente envie o token precedido pela palavra "Bearer " (ex: `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`).
    - Extração do Token: O cabeçalho é dividido pela string " ", e pegamos o segundo elemento, que é o token em si.
    - Verificação do Token: A função `jwt.verify(token, SEGREDO)` tenta decodificar o token usando a mesma chave secreta (`SEGREDO`) que foi usada para criá-lo.
        - Se o token for válido (não foi adulterado e não expirou), `jwt.verify` retorna o payload do token (que, no nosso caso, conterá o `id` do usuário).
        - Se o token for inválido (alterado, expirado, ou a chave secreta estiver incorreta), uma exceção é lançada.
    - Adição do `usuarioId` à Requisição: Se o token for válido, o `id` do usuário extraído do payload é adicionado como uma propriedade `usuarioId` ao objeto `req` (requisição). Isso permite que as rotas que usam este middleware acessem facilmente o ID do usuário autenticado.
    - Próximo Middleware/Controlador: `next()` é chamado para passar o controle para o próximo middleware na pilha ou para a função do controlador da rota que está sendo acessada.
    - Tratamento de Erros: Se ocorrer qualquer erro durante a verificação do token (por exemplo, se o token estiver ausente ou for inválido), o middleware retorna uma resposta com status `401 Unauthorized` e uma mensagem de erro.
- A Chave Secreta (`SEGREDO`):
    - A chave secreta é crucial para a segurança do JWT. Ela é usada tanto para assinar o token quanto para verificá-lo. Se alguém obtiver sua chave secreta, poderá forjar tokens válidos.
    - Em um ambiente de produção, é FUNDAMENTAL que esta chave seja forte, aleatória e mantida em segredo. Ela nunca deve ser hardcoded diretamente no código fonte. Em vez disso, deve ser carregada de variáveis de ambiente (usando pacotes como `dotenv`).

Este middleware `auth` será aplicado em rotas que exigem que o usuário esteja logado, garantindo que apenas usuários autenticados possam acessá-las.

Capítulo 6 — Rotas de Autenticação

As rotas de autenticação são o ponto de entrada para que os usuários possam se cadastrar e fazer login em nosso sistema. Elas lidam com a criação de novas contas e a validação de credenciais existentes, emitindo tokens JWT para acesso seguro.

Arquivo Completo:

TypeScript

```
import { Router } from "express";
import bcrypt from "bcryptjs"; // Para criptografar senhas
```

```
import jwt from "jsonwebtoken"; // Para gerar tokens JWT
import db from "../database/database"; // Importa a instância do banco de dados

const router = Router(); // Cria um novo roteador Express

// Define a mesma chave secreta usada no middleware de autenticação.
// Em produção, deve ser carregada de variáveis de ambiente.
const SEGREDO = "segredo_super_simples_deve_ser_trocado";

/**
 * Rota para cadastro de novos usuários.
 * Método: POST
 * Endpoint: /auth/cadastro
 * Corpo da requisição: { nome: string, email: string, senha: string }
 */
router.post("/cadastro", async (req, res) => {
  const { nome, email, senha } = req.body; // Extrai os dados do corpo da requisição

  // Verifica se o email já está cadastrado no banco de dados.
  const usuarioExistente = db
    .prepare("SELECT id FROM usuarios WHERE email = ?")
    .get(email);

  // Se o email já existe, retorna um erro.
  if (usuarioExistente) {
    return res.status(409).json({ erro: "Email já cadastrado" }); // 409 Conflict
  }

  // Criptografa a senha do usuário usando bcrypt.
  const hash = await bcrypt.hash(senha, 8); // O '8' é o custo computacional (salt rounds)

  try {
    // Insere o novo usuário no banco de dados com o nome, email e senha criptografada.
    const result = db
      .prepare("INSERT INTO usuarios (nome, email, senha) VALUES (?, ?, ?)")
      .run(nome, email, hash);

    // Retorna o ID do usuário recém-criado.
    res.status(201).json({ id: result.lastInsertRowid }); // 201 Created
  } catch (error) {
    // Em caso de qualquer outro erro durante a inserção, retorna um erro genérico.
    console.error("Erro no cadastro:", error);
    res.status(500).json({ erro: "Erro interno no servidor" });
  }
});

/**
 * Rota para login de usuários existentes.
 * Método: POST
 * Endpoint: /auth/login
 * Corpo da requisição: { email: string, senha: string }
 */
router.post("/login", async (req, res) => {
  const { email, senha } = req.body; // Extrai email e senha do corpo da requisição

  // Busca o usuário no banco de dados pelo email fornecido.
  const usuario = db
    .prepare("SELECT id, senha FROM usuarios WHERE email = ?") // Seleciona id e senha para comparação
    .get(email);

  // Se o usuário não for encontrado, retorna um erro.
  if (!usuario) {
    return res.status(404).json({ erro: "Usuário não encontrado" }); // 404 Not Found
  }

  // Compara a senha fornecida com o hash armazenado no banco de dados.
  const senhaCorreta = await bcrypt.compare(senha, usuario.senha);

  // Se as senhas não corresponderem, retorna um erro.
  if (!senhaCorreta) {
    return res.status(401).json({ erro: "Senha inválida" }); // 401 Unauthorized
  }

  // Se as credenciais forem válidas, gera um token JWT.
  // O payload do token contém o ID do usuário.
  const token = jwt.sign({ id: usuario.id }, SEGREDO, { expiresIn: "1h" }); // Token expira em 1 hora
```

```
  // Retorna o token JWT para o cliente.
  res.json({ token });
});

export default router; // Exporta o roteador para ser usado no app.ts
```

Explicação Detalhada: Cadastro e Login

- Rota de Cadastro (`/cadastro` - POST):
  1. Extração de Dados: O nome, email e senha são extraídos do corpo da requisição (`req.body`).
  2. Verificação de Email Existente: Antes de criar um novo usuário, consultamos o banco de dados para garantir que o email fornecido não esteja já em uso (`SELECT id FROM usuarios WHERE email = ?`). Se um usuário com esse email for encontrado, retornamos um erro `409 Conflict`.
  3. Criptografia da Senha: A senha fornecida pelo usuário é criptografada usando `bcrypt.hash()`. Isso garante que a senha nunca seja armazenada em texto plano.
  4. Inserção no Banco de Dados: O novo usuário é inserido na tabela `usuarios` com seu nome, email e a senha criptografada.
  5. Resposta: Em caso de sucesso, retornamos um status `201 Created` e o ID do novo usuário. Em caso de erro, retornamos um status `500 Internal Server Error`.
- Rota de Login (`/login` - POST):
  1. Extração de Credenciais: O email e a senha são extraídos do corpo da requisição.
  2. Busca do Usuário: Consultamos o banco de dados para encontrar um usuário com o email fornecido (`SELECT id, senha FROM usuarios WHERE email = ?`). Selecionamos o `id` e a `senha` (hash) do usuário.
  3. Verificação de Usuário: Se nenhum usuário for encontrado com o email fornecido, retornamos um erro `404 Not Found`.
  4. Comparação de Senhas: Utilizamos `bcrypt.compare(senha, usuario.senha)` para comparar a senha que o usuário digitou com o hash armazenado no banco de dados. Essa função é segura e leva em conta o salt usado durante o hashing.
  5. Verificação de Senha: Se as senhas não corresponderem, retornamos um erro `401 Unauthorized`.
  6. Geração de Token JWT: Se as credenciais estiverem corretas, geramos um token JWT usando `jwt.sign()`.
     - O primeiro argumento é o `payload` do token, que neste caso contém o `id` do usuário.
     - O segundo argumento é a `SEGREDO` (a chave secreta).
     - O terceiro argumento é um objeto de opções, onde definimos `expiresIn: "1h"` para que o token expire em uma hora. Isso é uma boa prática de segurança.

7. Resposta: Retornamos o token JWT gerado em um objeto JSON. O cliente (aplicativo mobile) deve armazenar este token e enviá-lo em requisições futuras para acessar recursos protegidos.

Com estas rotas, os usuários podem agora criar suas contas e autenticar-se em nosso sistema.

Capítulo 7 — Rotas de Usuário

Neste capítulo, implementaremos rotas para acessar informações específicas do usuário. A principal rota será a de "perfil", que permitirá ao usuário autenticado visualizar seus próprios dados.

Arquivo Completo:

TypeScript

```
import { Router } from "express";
import db from "../database/database"; // Importa a instância do banco de dados
import { auth } from "../middlewares/auth"; // Importa o middleware de autenticação

const router = Router(); // Cria um novo roteador Express

/**
 * Rota para obter o perfil do usuário autenticado.
 * Método: GET
 * Endpoint: /usuarios/perfil
 * Requer autenticação (token JWT).
 */
router.get("/perfil", auth, (req, res) => {
  // O middleware 'auth' já adicionou o ID do usuário autenticado em req.usuarioId.
  // Agora, buscamos os dados do usuário no banco de dados usando esse ID.
  const usuario = db
    .prepare("SELECT id, nome, email FROM usuarios WHERE id = ?") // Seleciona apenas os campos relevantes
para o perfil
    .get(req.usuarioId); // Usa o ID do usuário obtido do token

  // Se por algum motivo o usuário não for encontrado (o que não deveria acontecer se o token for válido),
  // retornamos um erro.
  if (!usuario) {
    return res.status(404).json({ erro: "Usuário não encontrado" });
  }

  // Retorna os dados do perfil do usuário.
  res.json(usuario);
});

export default router; // Exporta o roteador para ser usado no app.ts
```

Explicação Detalhada: Perfil do Usuário

- Rota `/perfil` (GET):
  1. Aplicação do Middleware `auth`: A rota `router.get("/perfil", auth, ...)` indica que o middleware `auth` será executado antes da função do controlador (a

função `(req, res) => {...}`). Isso significa que apenas usuários que apresentarem um token JWT válido poderão acessar esta rota.
2. Acesso ao `req.usuarioId`: Graças ao middleware `auth`, o ID do usuário autenticado está disponível em `req.usuarioId`.
3. Consulta ao Banco de Dados: Utilizamos este `req.usuarioId` para buscar as informações do usuário na tabela `usuarios`. Selecionamos apenas `id`, `nome` e `email`, pois são os dados relevantes para um perfil.
4. Tratamento de Erro (Inesperado): Incluímos uma verificação `if (!usuario)` por robustez. Se, por algum motivo, o usuário com o ID do token não for encontrado no banco de dados, retornamos um erro `404`. No entanto, em um fluxo normal, isso não deve ocorrer se o token for válido.
5. Resposta: Se o usuário for encontrado, seus dados (`id`, `nome`, `email`) são retornados em formato JSON.

Essa rota é um exemplo simples de como proteger endpoints e acessar informações contextuais do usuário logado.

Capítulo 8 — Rotas de Reservas

Neste capítulo, implementaremos as rotas essenciais para gerenciar as reservas de salas. Incluiremos a funcionalidade de criar uma nova reserva e de listar todas as reservas feitas por um usuário autenticado.

Arquivo Completo:

TypeScript

```
import { Router } from "express";
import db from "../database/database"; // Importa a instância do banco de dados
import { auth } from "../middlewares/auth"; // Importa o middleware de autenticação

const router = Router(); // Cria um novo roteador Express

/**
 * Rota para criar uma nova reserva.
 * Método: POST
 * Endpoint: /reservas/
 * Requer autenticação (token JWT).
 * Corpo da requisição: { sala: string, data: string }
 */
router.post("/", auth, (req, res) => {
  // Extrai os dados da sala e data do corpo da requisição.
  const { sala, data } = req.body;

  // O ID do usuário autenticado está disponível em req.usuarioId graças ao middleware 'auth'.
  const usuarioId = req.usuarioId;

  // Validação básica: verifica se os campos obrigatórios foram enviados.
  if (!sala || !data) {
    return res.status(400).json({ erro: "Sala e data são obrigatórias" }); // 400 Bad Request
  }

  try {
    // Insere a nova reserva no banco de dados.
    // O 'usuario_id' é preenchido com o ID do usuário autenticado.
    const result = db
      .prepare("INSERT INTO reservas (sala, data, usuario_id) VALUES (?, ?, ?)")
      .run(sala, data, usuarioId);
```

```
    // Retorna o ID da reserva recém-criada.
    res.status(201).json({ id: result.lastInsertRowid }); // 201 Created
  } catch (error) {
    // Em caso de erro na inserção (ex: problema de integridade referencial, embora improvável com a
validação prévia).
    console.error("Erro ao criar reserva:", error);
    res.status(500).json({ erro: "Erro interno no servidor ao criar reserva" });
  }
});

/**
 * Rota para listar todas as reservas de um usuário autenticado.
 * Método: GET
 * Endpoint: /reservas/
 * Requer autenticação (token JWT).
 */
router.get("/", auth, (req, res) => {
  // O ID do usuário autenticado está disponível em req.usuarioId.
  const usuarioId = req.usuarioId;

  try {
    // Busca todas as reservas associadas ao ID do usuário autenticado.
    const reservas = db
      .prepare("SELECT id, sala, data FROM reservas WHERE usuario_id = ?") // Seleciona campos relevantes
      .all(usuarioId); // O método .all() retorna um array com todas as linhas que correspondem ao
critério.

    // Retorna a lista de reservas do usuário.
    res.json(reservas);
  } catch (error) {
    // Em caso de erro na consulta.
    console.error("Erro ao listar reservas:", error);
    res.status(500).json({ erro: "Erro interno no servidor ao listar reservas" });
  }
});

export default router; // Exporta o roteador para ser usado no app.ts
```

Explicação Detalhada: Criando e Listando Reservas

- Rota de Criação de Reserva (`/` - POST):
  1. Proteção por Autenticação: A rota é protegida pelo middleware `auth`, garantindo que apenas usuários logados possam criar reservas.
  2. Extração de Dados: Os campos `sala` e `data` são extraídos do corpo da requisição.
  3. Validação Básica: Verificamos se `sala` e `data` foram fornecidos. Se não, retornamos um erro `400 Bad Request`.
  4. Acesso ao `req.usuarioId`: O ID do usuário autenticado é obtido de `req.usuarioId`.
  5. Inserção no Banco de Dados: Uma nova reserva é criada na tabela `reservas`, associando a sala, a data e o `usuario_id` (do usuário logado).
  6. Resposta: Em caso de sucesso, retornamos um status `201 Created` e o ID da reserva. Em caso de erro, um status `500 Internal Server Error`.
- Rota de Listagem de Reservas (`/` - GET):
  1. Proteção por Autenticação: Assim como a rota de criação, esta também requer autenticação.

2. Acesso ao `req.usuarioId`: O ID do usuário é obtido de `req.usuarioId`.
3. Consulta ao Banco de Dados: Buscamos na tabela `reservas` por todas as entradas onde o `usuario_id` corresponde ao ID do usuário autenticado. Selecionamos apenas `id`, `sala` e `data` para a resposta.
4. Resposta: Retornamos um array contendo todas as reservas do usuário. Se não houver reservas, o array estará vazio. Em caso de erro, um status `500 Internal Server Error`.

Com estas rotas, o sistema agora permite que usuários criem e visualizem suas próprias reservas de forma segura.

Capítulo 9 — Aplicativo Mobile (Expo)

Agora vamos construir a interface do usuário e a lógica de comunicação para nosso aplicativo mobile usando Expo (React Native).

(Imagem: Capturas de tela simuladas das telas de Login, Cadastro e Home do aplicativo mobile. As telas devem ser limpas e funcionais.)

Configuração da API:

No aplicativo mobile, precisamos definir o endereço da nossa API. Crie ou edite o arquivo `app/lib/services/api.ts`:

TypeScript

```
// app/lib/services/api.ts

// Substitua 'SEU_IP' pelo IP local da sua máquina onde o backend está rodando.
// Ex: export const API_URL = "http://192.168.1.10:3000";
export const API_URL = "http://SEU_IP:3000";
```

Tela de Login (Exemplo de Lógica):

JavaScript

```
// Em um componente de tela de Login (ex: app/screens/LoginScreen.tsx)

import React, { useState } from 'react';
import { View, TextInput, Button, Alert } from 'react-native';
import { API_URL } from '../lib/services/api'; // Importa a URL da API

export default function LoginScreen({ navigation }) {
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');

  async function fazerLogin() {
```

```
    try {
      const response = await fetch(`${API_URL}/auth/login`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({ email, senha }),
      });

      const data = await response.json();

      if (response.ok && data.token) {
        // Se o login for bem-sucedido, navega para a tela Home e passa o token.
        // Em um app real, você armazenaria o token de forma segura (ex: AsyncStorage).
        navigation.navigate('Home', { token: data.token });
      } else {
        Alert.alert('Erro no Login', data.erro || 'Verifique suas credenciais.');
      }
    } catch (error) {
      console.error("Erro ao fazer login:", error);
      Alert.alert('Erro de Rede', 'Não foi possível conectar ao servidor.');
    }
  }

  return (
    <View>
      <TextInput placeholder="Email" value={email} onChangeText={setEmail} />
      <TextInput placeholder="Senha" secureTextEntry value={senha} onChangeText={setSenha} />
      <Button title="Login" onPress={fazerLogin} />
      {/* Botão para navegar para a tela de cadastro */}
      <Button title="Cadastrar" onPress={() => navigation.navigate('Cadastro')} />
    </View>
  );
}
```

Tela de Cadastro (Exemplo de Lógica):

JavaScript

```
// Em um componente de tela de Cadastro (ex: app/screens/CadastroScreen.tsx)

import React, { useState } from 'react';
import { View, TextInput, Button, Alert } from 'react-native';
import { API_URL } from '../lib/services/api';

export default function CadastroScreen({ navigation }) {
  const [nome, setNome] = useState('');
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');

  async function cadastrarUsuario() {
    try {
      const response = await fetch(`${API_URL}/auth/cadastro`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({ nome, email, senha }),
      });

      const data = await response.json();

      if (response.ok) {
        Alert.alert('Cadastro Realizado', 'Usuário criado com sucesso!');
        navigation.goBack(); // Volta para a tela anterior (Login) após o cadastro
```

```
      } else {
        Alert.alert('Erro no Cadastro', data.erro || 'Tente novamente.');
      }
    } catch (error) {
      console.error("Erro ao cadastrar:", error);
      Alert.alert('Erro de Rede', 'Não foi possível conectar ao servidor.');
    }
  }

  return (
    <View>
      <TextInput placeholder="Nome Completo" value={nome} onChangeText={setNome} />
      <TextInput placeholder="Email" value={email} onChangeText={setEmail} />
      <TextInput placeholder="Senha" secureTextEntry value={senha} onChangeText={setSenha} />
      <Button title="Cadastrar" onPress={cadastrarUsuario} />
    </View>
  );
}
```

Tela Principal (Home) - Exemplo de Lógica:

Esta tela pode exibir as reservas do usuário e oferecer a opção de criar novas.

JavaScript

```
// Em um componente de tela Home (ex: app/screens/HomeScreen.tsx)

import React, { useState, useEffect } from 'react';
import { View, Text, Button, FlatList, StyleSheet } from 'react-native';
import { API_URL } from '../lib/services/api';

export default function HomeScreen({ route, navigation }) {
  const { token } = route.params; // Recebe o token passado da tela de Login
  const [reservas, setReservas] = useState([]);
  const [loading, setLoading] = useState(true);

  async function carregarReservas() {
    try {
      setLoading(true);
      const response = await fetch(`${API_URL}/reservas/`, {
        method: 'GET',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${token}`, // Envia o token no cabeçalho Authorization
        },
      });

      if (response.ok) {
        const data = await response.json();
        setReservas(data);
      } else {
        // Se o token for inválido ou expirado, pode ser necessário redirecionar para o login.
        const errorData = await response.json();
        if (response.status === 401) {
            Alert.alert("Autenticação Inválida", "Seu token expirou ou é inválido. Por favor, faça login
novamente.");
            navigation.navigate('Login');
        } else {
            Alert.alert("Erro", errorData.erro || "Não foi possível carregar as reservas.");
        }
      }
    } catch (error) {
      console.error("Erro ao carregar reservas:", error);
      Alert.alert('Erro de Rede', 'Não foi possível conectar ao servidor.');
```

```
    } finally {
      setLoading(false);
    }
  }

  // Carrega as reservas quando o componente é montado ou quando o token muda.
  useEffect(() => {
    carregarReservas();
  }, [token]); // Dependência no token para recarregar se ele mudar

  // Função para renderizar cada item da lista de reservas
  const renderItem = ({ item }) => (
    <View style={styles.reservaItem}>
      <Text>Sala: {item.sala}</Text>
      <Text>Data: {item.data}</Text>
    </View>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Suas Reservas</Text>
      {loading ? (
        <Text>Carregando...</Text>
      ) : reservas.length > 0 ? (
        <FlatList
          data={reservas}
          renderItem={renderItem}
          keyExtractor={(item) => item.id.toString()}
        />
      ) : (
        <Text>Você ainda não tem reservas.</Text>
      )}

      {/* Botão para criar uma nova reserva (assumindo que existe uma tela CreateReservaScreen) */}
      <Button title="Criar Nova Reserva" onPress={() => navigation.navigate('CreateReserva', { token })} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f5f5f5',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
    textAlign: 'center',
  },
  reservaItem: {
    backgroundColor: 'white',
    padding: 15,
    marginBottom: 10,
    borderRadius: 8,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 3, // Para Android
  },
});
```

Consumo Protegido (Header `Authorization`):

Ao consumir rotas protegidas no frontend (como `/reservas/` ou `/usuarios/perfil`), é fundamental enviar o token JWT no cabeçalho `Authorization`.

JavaScript

```javascript
// No fetch para uma rota protegida:
headers: {
  'Content-Type': 'application/json',
  'Authorization': `Bearer ${token}` // O token é enviado aqui
}
```

Capítulo 10 — Fluxo Completo de Ponta a Ponta

Com todas as peças prontas, vamos visualizar o fluxo completo de como um usuário interage com o sistema, desde o login até a visualização de suas reservas.

(Imagem: Um fluxograma detalhado mostrando a interação entre o Aplicativo Mobile e a API. Inclui os passos de login, obtenção do token, envio do token em requisições subsequentes e a resposta do backend.)

O Ciclo de Vida de uma Requisição (Exemplo: Ver reservas):

1. Usuário faz Login: No aplicativo mobile, o usuário insere seu email e senha e toca no botão "Login".
2. App envia Credenciais para API: O aplicativo mobile faz uma requisição `POST` para `/auth/login` na API, enviando as credenciais no corpo da requisição.
3. API Valida Credenciais e Gera Token: O backend recebe as credenciais, verifica no banco de dados se o usuário existe e se a senha está correta. Se tudo estiver certo, ele gera um token JWT contendo o ID do usuário e o retorna para o aplicativo.
4. App Armazena o Token: O aplicativo mobile recebe o token JWT e o armazena (idealmente de forma segura, como no `AsyncStorage`).
5. Usuário Navega para a Tela de Reservas: O usuário toca em um botão para ver suas reservas.
6. App Faz Requisição Protegida: O aplicativo mobile faz uma requisição `GET` para `/reservas/`. Crucialmente, ele inclui o token JWT armazenado no cabeçalho `Authorization` no formato `Bearer SEU_TOKEN_AQUI`.
7. API Valida o Token (Middleware `auth`): O middleware `auth` na API intercepta a requisição, extrai o token do cabeçalho e o verifica usando a chave secreta. Se o token for válido, ele extrai o `usuarioId` e o anexa ao objeto `req`.
8. API Busca Reservas do Usuário: A rota `/reservas/` (protegida) utiliza o `req.usuarioId` para consultar o banco de dados e buscar todas as reservas associadas a esse usuário.
9. API Retorna os Dados: O backend envia a lista de reservas do usuário de volta para o aplicativo mobile.
10. App Exibe as Reservas: O aplicativo mobile recebe os dados e os exibe na tela para o usuário.

Este fluxo demonstra a comunicação contínua e segura entre o frontend e o backend, onde o JWT atua como a chave para desbloquear o acesso aos dados do usuário.

Capítulo 11 — Testes Básicos

Para verificar se nossa API está funcionando corretamente, podemos usar ferramentas como `curl` para simular requisições.

Testando o Login com `curl`:

Abra seu terminal e execute o seguinte comando:

Bash

```bash
curl -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@example.com","senha":"admin123"}'
```

Explicação do Comando:

- `curl`: A ferramenta de linha de comando para transferir dados.
- `-X POST`: Especifica o método HTTP como POST.
- `http://localhost:3000/auth/login`: O endpoint da API que estamos testando.
- `-H "Content-Type: application/json"`: Define o cabeçalho `Content-Type` para indicar que estamos enviando dados em formato JSON.
- `-d '{"email":"admin@example.com","senha":"admin123"}'`: Os dados a serem enviados no corpo da requisição.

Resultado Esperado:

Se o login for bem-sucedido, você receberá uma resposta JSON contendo o token JWT:

Json

```json
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."}
```

Você pode usar este token para testar outras rotas protegidas com `curl` ou enviá-lo do seu aplicativo mobile.

Capítulo 12 — Evolução e Próximos Passos

Este projeto é um excelente ponto de partida, mas o mundo do desenvolvimento de software é dinâmico e cheio de possibilidades de aprimoramento.

Sugestões de Melhoria:

- Banco de Dados mais Robusto: Para aplicações maiores, considere migrar do SQLite para bancos de dados mais escaláveis como PostgreSQL ou MySQL.
- ORM (Object-Relational Mapper): Utilize ORMs como Prisma ou Sequelize para abstrair as interações com o banco de dados, tornando o código mais limpo e seguro.
- Refresh Token: Implemente um sistema de refresh tokens para melhorar a segurança e a experiência do usuário, permitindo renovar o token de acesso sem que o usuário precise fazer login novamente a cada expiração.
- Containerização com Docker: Utilize Docker para empacotar sua aplicação e suas dependências, facilitando a implantação e garantindo ambientes consistentes.
- Testes Automatizados: Escreva testes unitários, de integração e end-to-end para garantir a robustez e a confiabilidade do seu código.
- Gerenciamento de Erros Aprimorado: Implemente um sistema de tratamento de erros mais sofisticado, com logs centralizados e mensagens de erro mais descritivas para o usuário.
- Validações Avançadas: Utilize bibliotecas como `Zod` ou `Yup` para validações de schema mais complexas no backend e no frontend.
- Notificações Push: Integre notificações push para alertar os usuários sobre atualizações importantes ou novas reservas.
- Notificações por Email: Envie emails de confirmação de reserva ou de boas-vindas.

Conclusão

Ao longo deste livro, você construiu um sistema completo de reservas, abrangendo o desenvolvimento de uma API robusta com Node.js e um aplicativo mobile interativo com Expo. Você aprendeu sobre a arquitetura de sistemas full-stack, a importância da preparação do ambiente, a configuração de bancos de dados, a implementação de autenticação segura com JWT e a comunicação entre frontend e backend.

Mais importante do que apenas o código, esperamos que você tenha compreendido o fluxo e a lógica por trás de cada etapa. O desenvolvimento de software é uma jornada contínua de aprendizado e aprimoramento. As habilidades e os conceitos que você adquiriu aqui servirão como uma base sólida para enfrentar desafios maiores e construir aplicações ainda mais complexas e inovadoras.

Continue praticando, explorando novas tecnologias e, acima de tudo, construindo!

Apêndice — Tipos TypeScript

Para melhorar a segurança e a robustez do nosso código TypeScript no backend, podemos estender as interfaces do Express para incluir propriedades customizadas, como o `usuarioId` que adicionamos no middleware de autenticação.

TypeScript

```
// Adicione este código em um arquivo .d.ts (ex: src/types/express.d.ts)

// Declaração de namespace para estender a interface Request do Express
declare namespace Express {
  // Estende a interface Request existente
  export interface Request {
    // Adiciona uma propriedade opcional usuarioId do tipo number
    // O '?' indica que a propriedade pode estar presente ou não.
    usuarioId?: number;
  }
}
```

Explicação:

- `declare namespace Express`: Isso informa ao TypeScript que estamos declarando ou modificando algo dentro do namespace `Express`.
- `export interface Request`: Estamos estendendo a interface `Request` que o Express já define.
- `usuarioId?: number;`: Adicionamos uma nova propriedade chamada `usuarioId`. O ponto de interrogação `?` a torna opcional, o que é importante porque nem todas as requisições passarão pelo middleware de autenticação. Quando o middleware `auth` é executado com sucesso, ele garante que `usuarioId` estará presente.

Ao adicionar este arquivo ao seu projeto TypeScript, você terá autocompletar e verificação de tipo para `req.usuarioId` em suas rotas, tornando seu código mais seguro e fácil de manter.

Anexo 1 - Tela de reserva